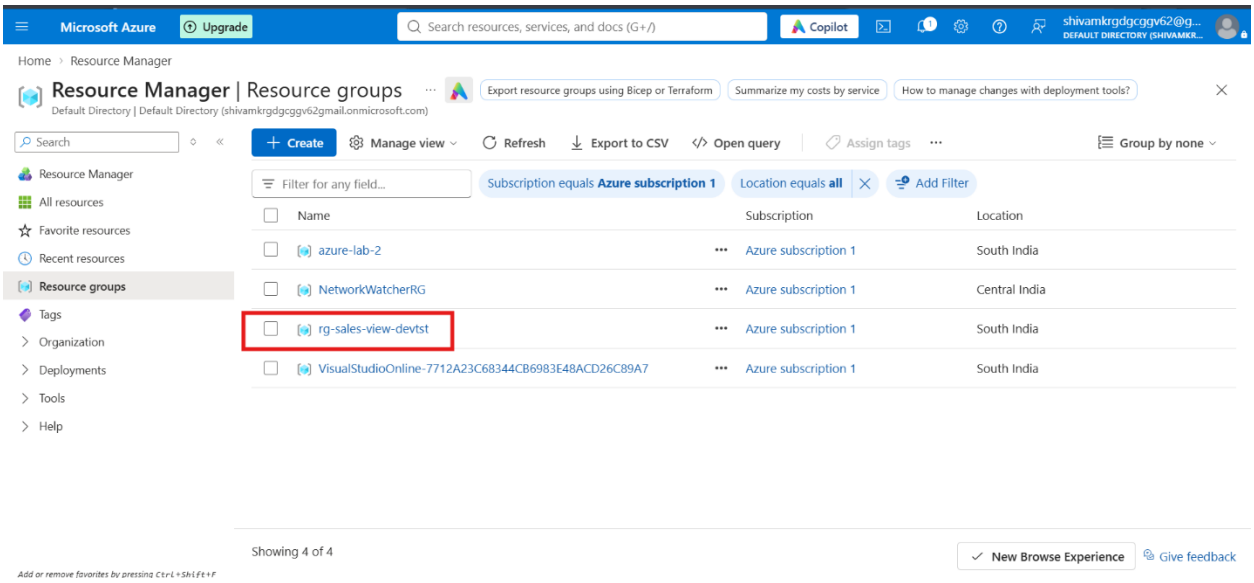
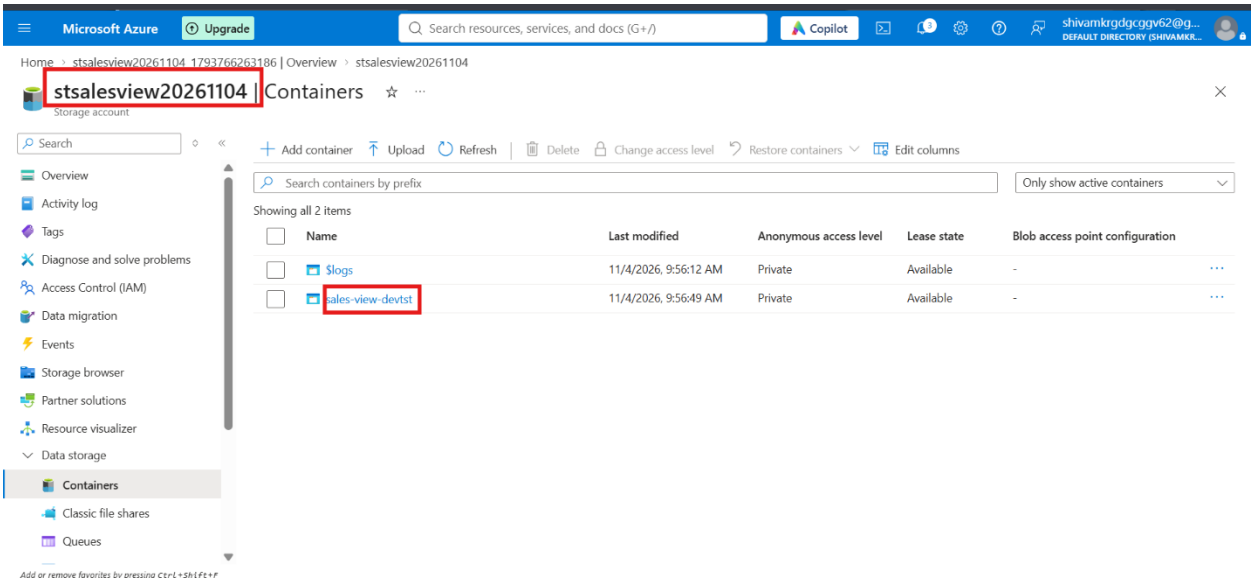
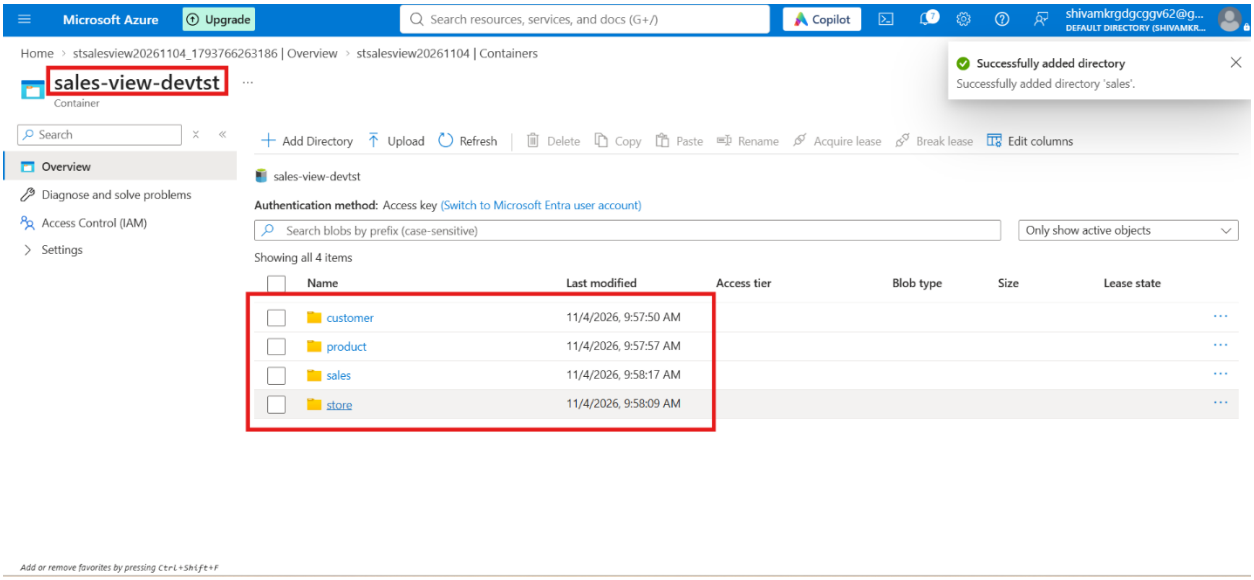
ADF Assignment

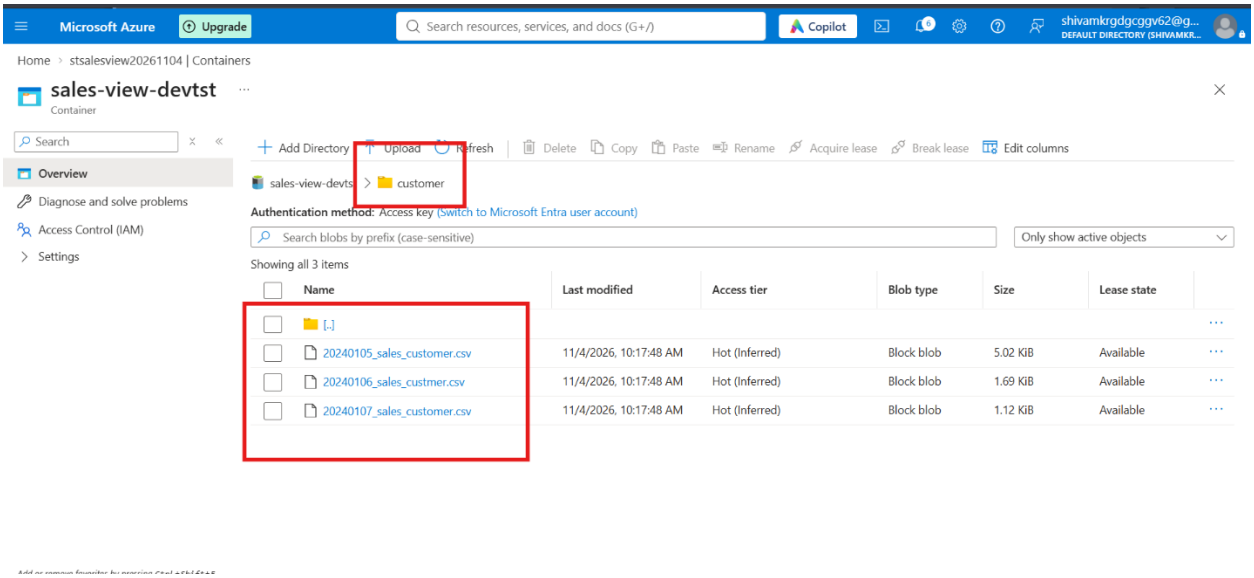
1) Create a container in ADLS with the project name as sales_view_devtst

- Create a folder for customer, product, store, and sales - Upload the files in sequence to make it work in real-time.



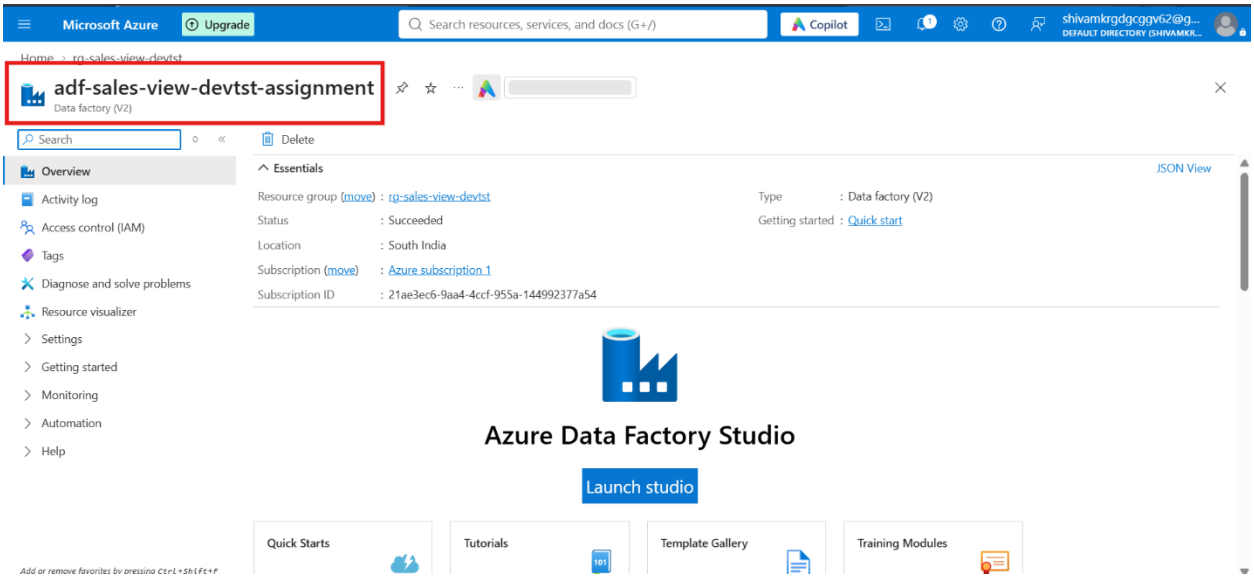


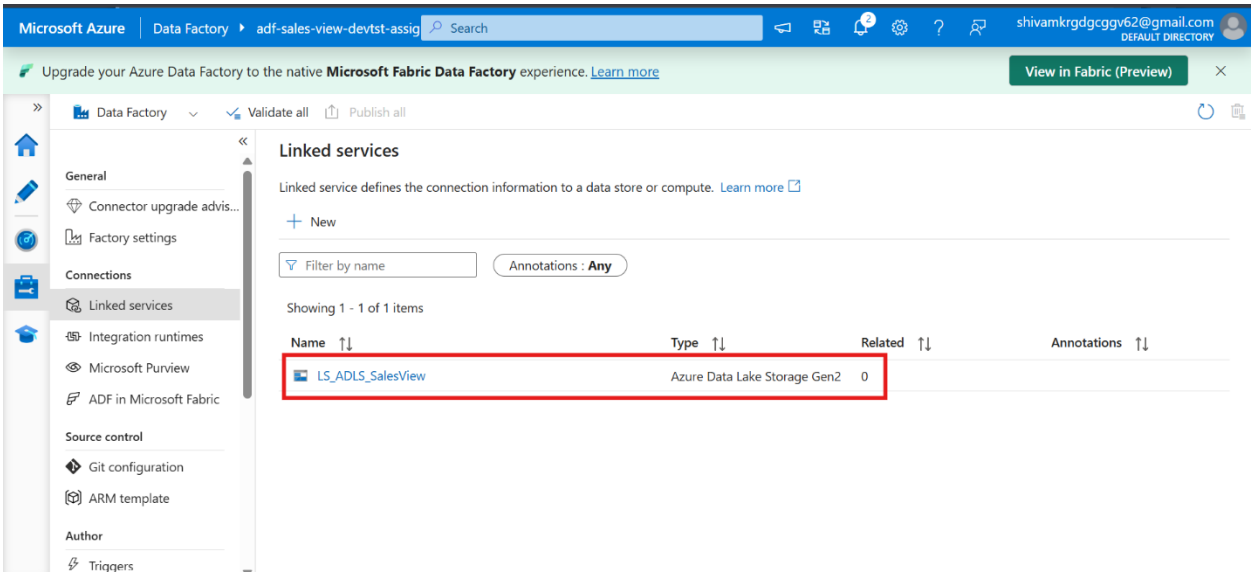


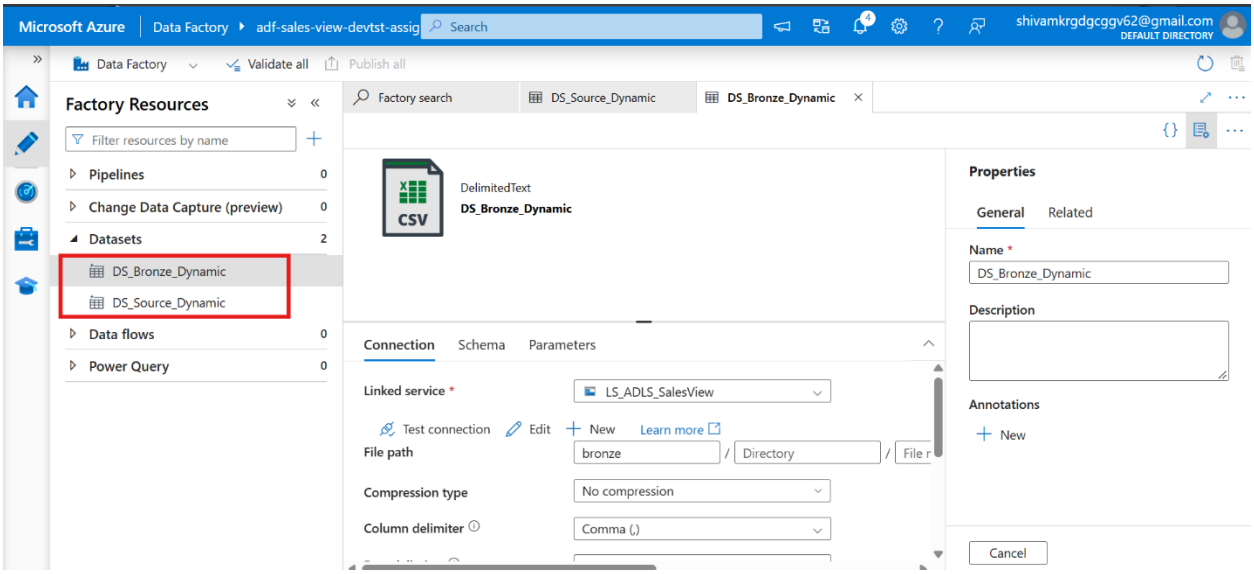


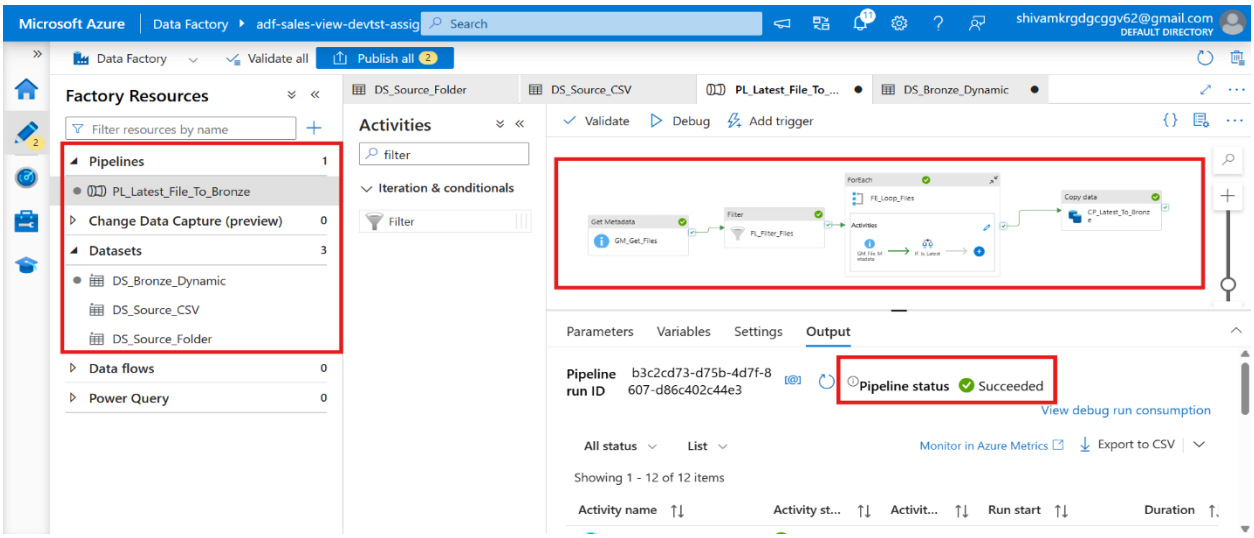
2) Create an ADF Pipeline to get the latest modified files the folders of the ADLS

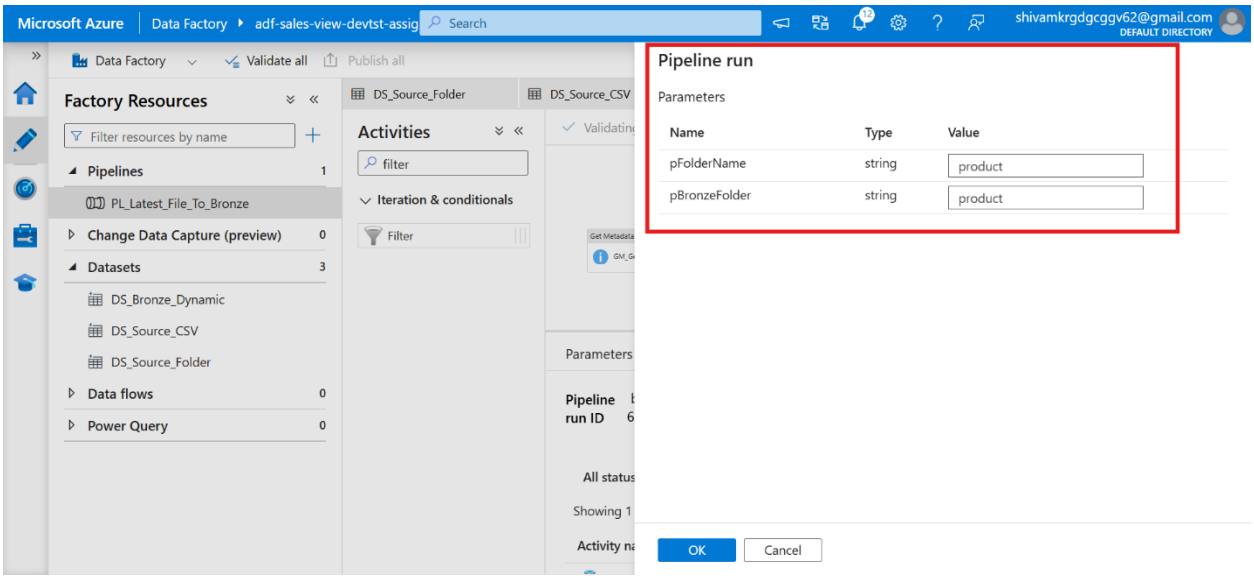
- Parameterize the Pipeline to work dynamically.
- Values need to be passed through parameters and it should work for all day's file


Microsoft Azure
Upgrade
Search resources, services, and docs (G+/)
Copilot
shivamkrgdgcggv62@g...
DEFAULT DIRECTORY (SHIVAMKR...
Home > rg-sales-view-devtst
adf-sales-view-devtst-assignment
Data factory (V2)
Search
Delete
Overview
Activity log
Access control (IAM)
Tags
Diagnose and solve problems
Resource visualizer
Settings
Getting started
Monitoring
Automation
Help
Essentials
JSON View
Resource group (move) : rg-sales-view-devtst
Status : Succeeded
Location : South India
Subscription (move) : Azure subscription 1
Subscription ID : 21ae3ec6-9aa4-4ccf-955a-144992377a54
Type : Data factory (V2)
Getting started : Quick start
Azure Data Factory Studio
Launch studio
Quick Starts
Tutorials
Template Gallery
Training Modules
Add or remove favorites by pressing Ctrl+Shift+F


Microsoft Azure
Data Factory
adf-sales-view-devtst-assig
Search
shivamkrgdgcggv62@gmail.com
DEFAULT DIRECTORY
Upgrade your Azure Data Factory to the native Microsoft Fabric Data Factory experience. Learn more
View in Fabric (Preview)
Data Factory
Validate all
Publish all
General
Connector upgrade advis...
Factory settings
Connections
Linked services
Integration runtimes
Microsoft Purview
ADF in Microsoft Fabric
Source control
Git configuration
ARM template
Author
Triggers
Linked services
Linked service defines the connection information to a data store or compute. Learn more
New
Filter by name
Annotations : Any
Showing 1 - 1 of 1 items
Name
Type
Related
Annotations
LS_ADLS_SalesView
Azure Data Lake Storage Gen2
0

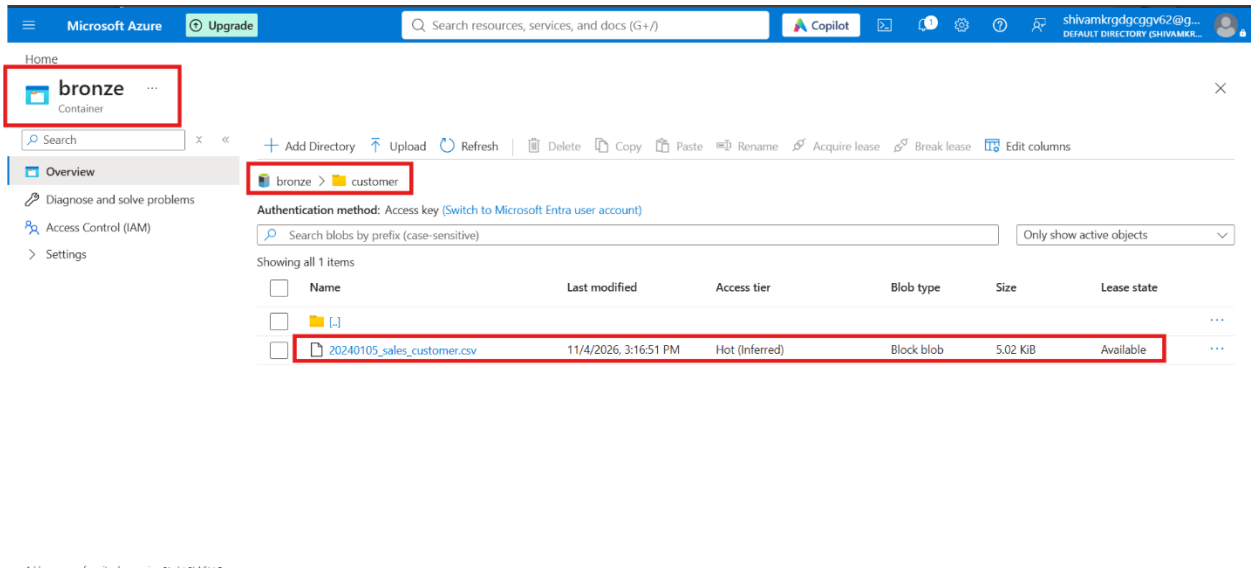

Microsoft Azure
Data Factory
adf-sales-view-devtst-assig
Search
shivamkrgdgcggv62@gmail.com
DEFAULT DIRECTORY
Validate all
Publish all
Factory Resources
Filter resources by name
Pipelines 0
Change Data Capture (preview) 0
Datasets 2
DS_Bronze_Dynamic
DS_Source_Dynamic
Data flows 0
Power Query 0
Factory search
DS_Source_Dynamic
DS_Bronze_Dynamic
DelimitedText
DS_Bronze_Dynamic
Properties
General
Related
Name *
DS_Bronze_Dynamic
Description
Annotations
New
Connection
Schema
Parameters
Linked service *
LS_ADLS_SalesView
Test connection
Edit
New
Learn more
File path
bronze
Directory
Compression type
No compression
Column delimiter
Comma (,)
Cancel

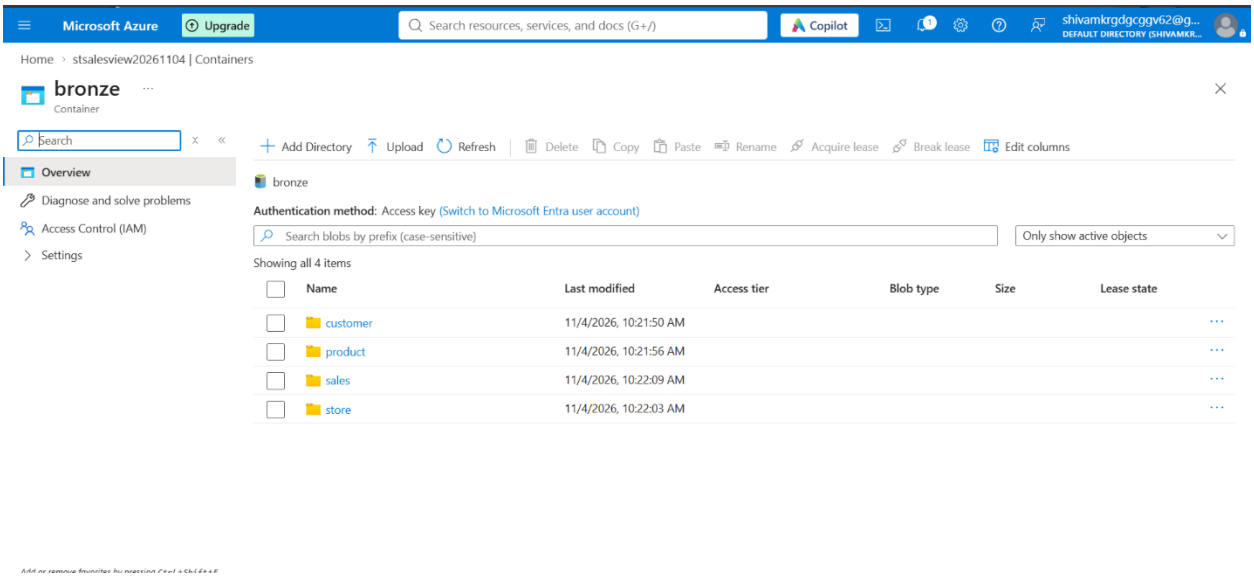

Microsoft Azure
Data Factory
adf-sales-view-devtst-assig
Search
shivamkrgdgcggv62@gmail.com
DEFAULT DIRECTORY
Validate all
Publish all 2
Factory Resources
Filter resources by name
Pipelines 1
PL_Latest_File_To_Bronze
Change Data Capture (preview) 0
Datasets 3
DS_Bronze_Dynamic
DS_Source_CSV
DS_Source_Folder
Data flows 0
Power Query 0
DS_Source_Folder
DS_Source_CSV
PL_Latest_File_To_...
DS_Bronze_Dynamic
Activities
filter
Iteration & conditionals
Filter
Validate
Debug
Add trigger
Parameters
Variables
Settings
Output
Pipeline run ID
b3c2cd73-d75b-4d7f-8607-d86c402c44e3
Pipeline status
Succeeded
View debug run consumption
All status
List
Monitor in Azure Metrics
Export to CSV
Showing 1 - 12 of 12 items
Activity name
Activity st...
Activit...
Run start
Duration


Microsoft Azure
Data Factory
adf-sales-view-devtst-assig
Search
shivamkrgdgcggv62@gmail.com
DEFAULT DIRECTORY
Validate all
Publish all
Factory Resources
Filter resources by name
Pipelines 1
PL_Latest_File_To_Bronze
Change Data Capture (preview) 0
Datasets 3
DS_Bronze_Dynamic
DS_Source_CSV
DS_Source_Folder
Data flows 0
Power Query 0
DS_Source_Folder
DS_Source_CSV
Activities
filter
Iteration & conditionals
Filter
Pipeline run
Parameters
Name
Type
Value
pFolderName
string
product
pBronzeFolder
string
product
OK
Cancel

3) Create a Bronze/sales view/ (Bronze – container)

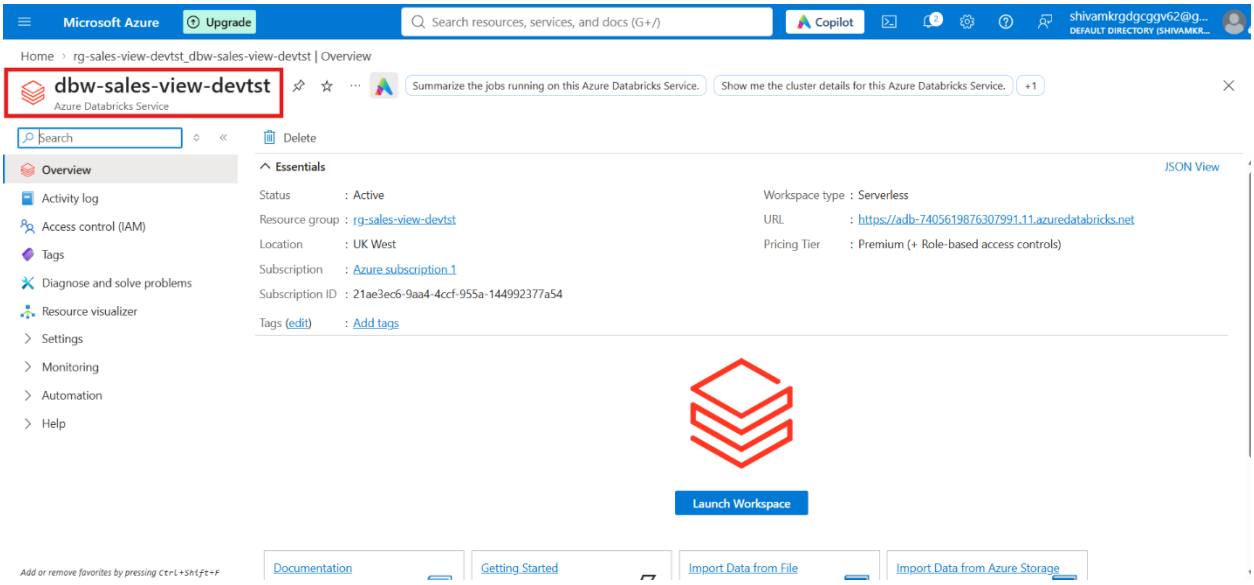
- Subfolders-> customer, product, store, sales, and store the raw data copied from the ADF pipeline

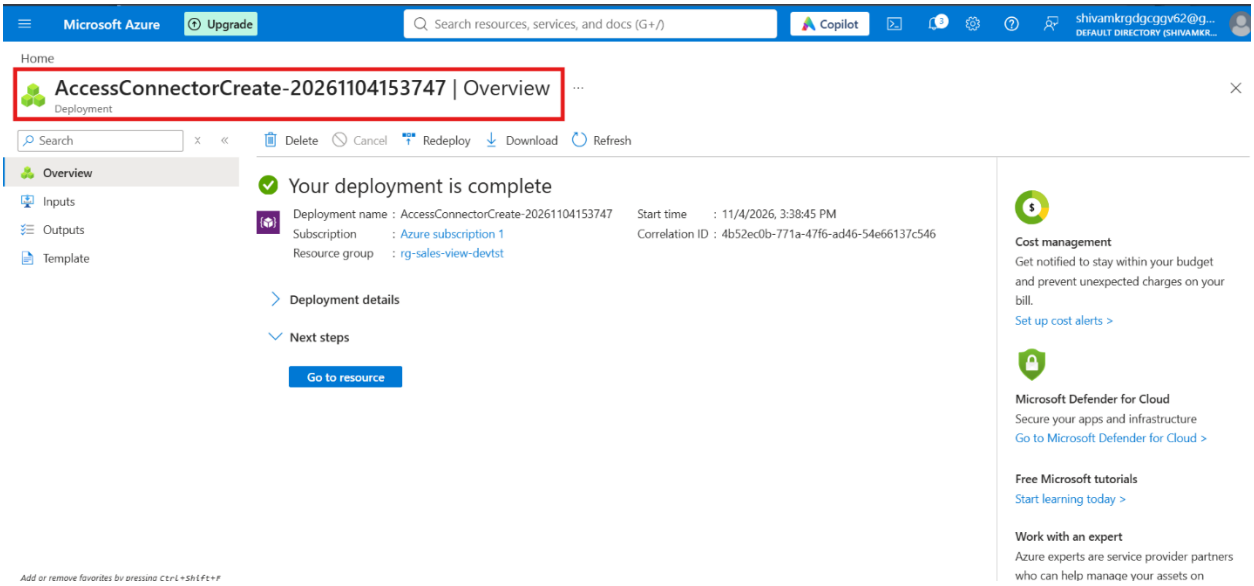


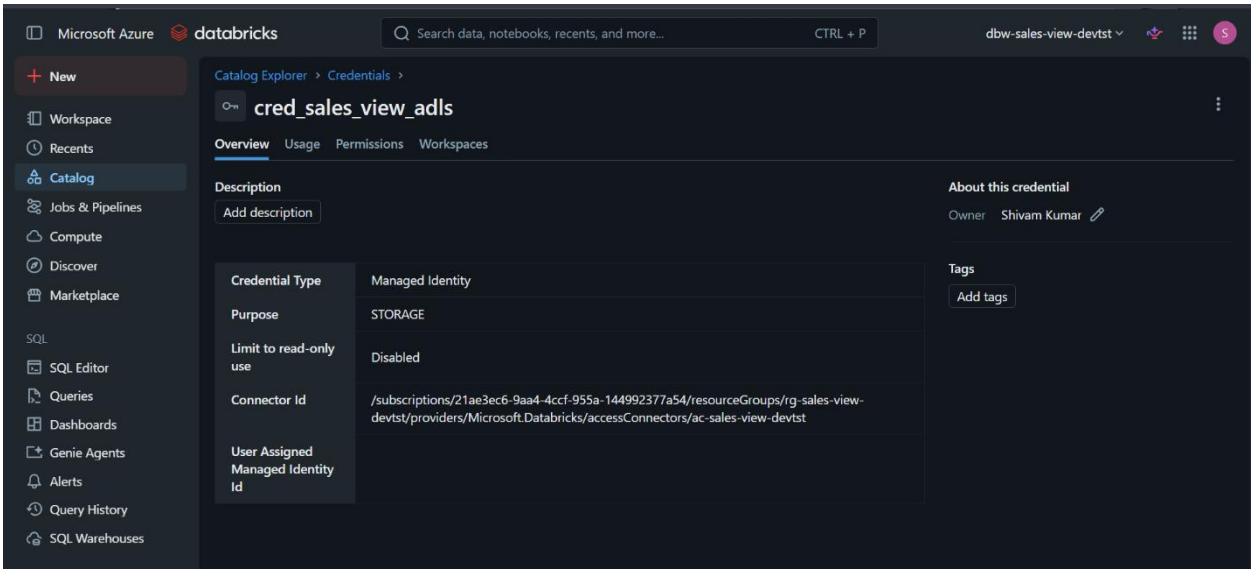


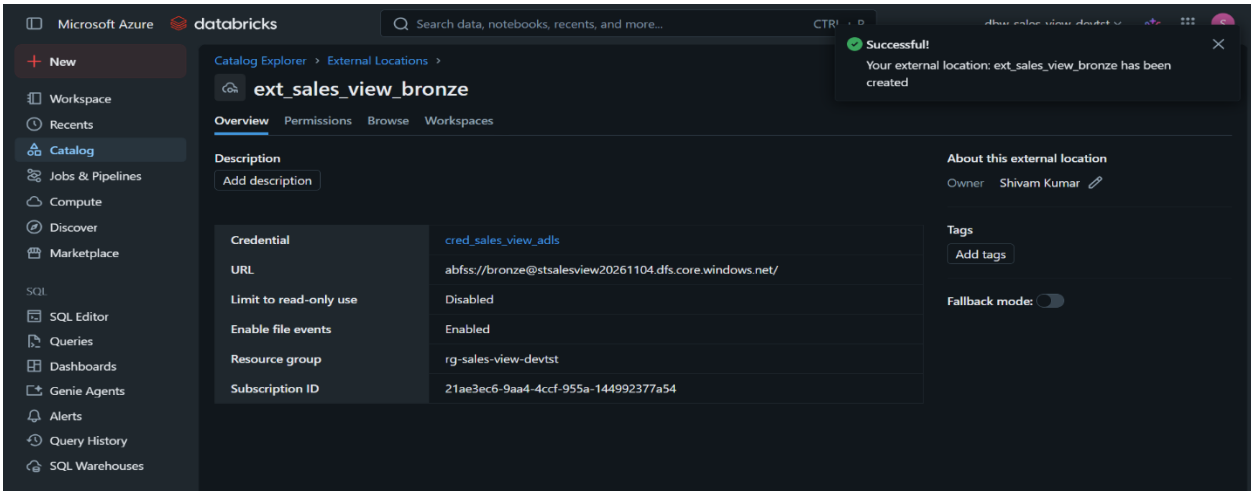
4) Customer file

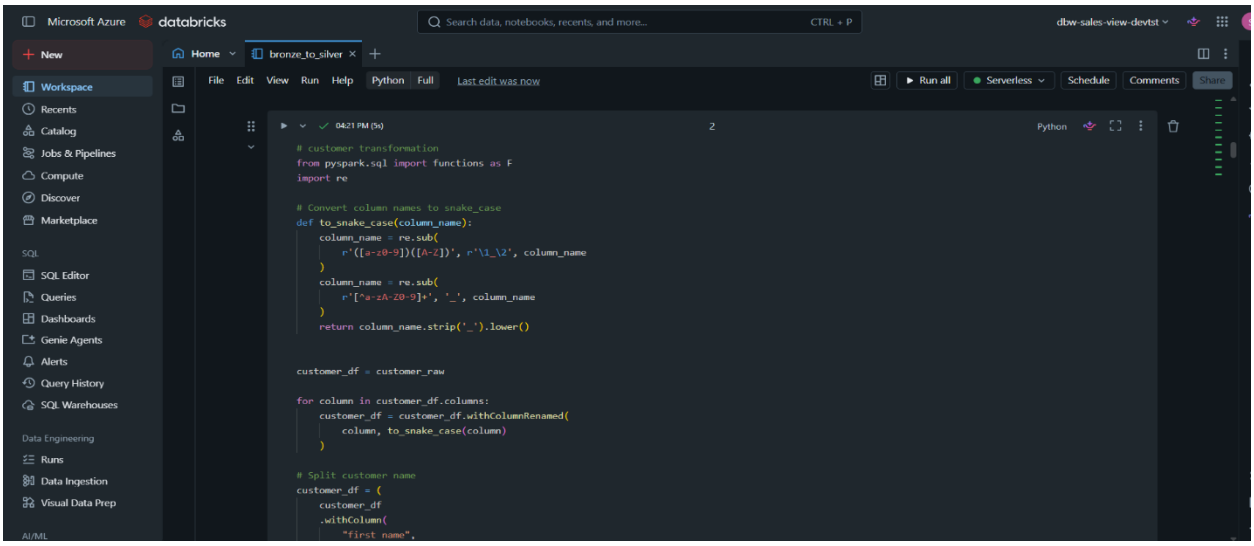
- All the column headers should be in snake case in lower case (By using UDF function dynamically works for all camel case to snake case)
- By using the "Name" column split by " " and create two columns first_name and last_name
- Create column domain and extract from email columns Ex: Email = "josephrice131@slingacademy.com" domain="slingacademy"
- Create a column ‘gender where male = "M" and Female="F"
- From Joining date create two columns date and time by splitting based on " " delimiter.
- Date column should be on "yyyy-MM-dd" format.
- Create a column expenditure-status, based on spent column is spent below 200 column value is "MINIMUM" else "MAXIMUM"
- Write based on upsert [table_name: customer] (in silver layer path is silver/sales_view/tablename/{delta pearquet}

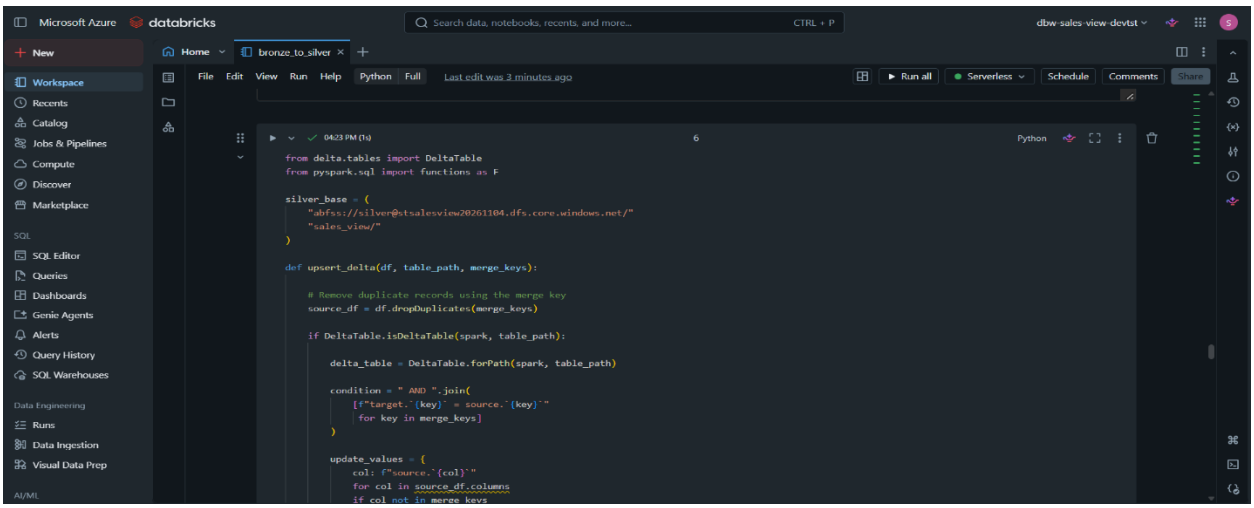


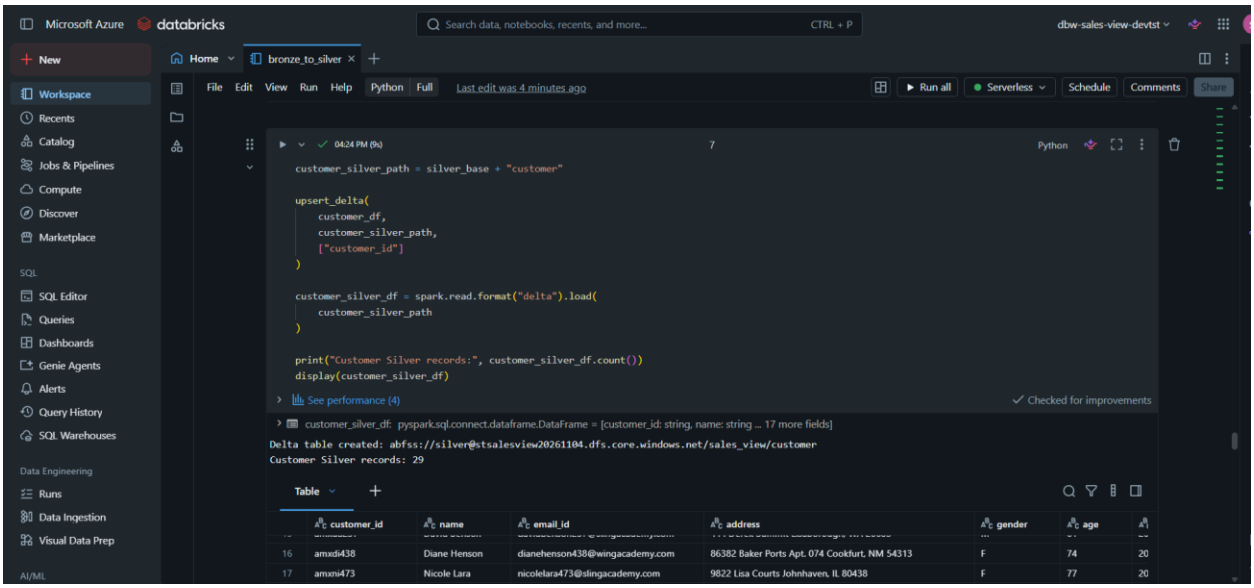

Microsoft Azure
Upgrade
Search resources, services, and docs (G+/)
Copilot
Home
AccessConnectorCreate-20261104153747 | Overview
Deployment
Search
Overview
Inputs
Outputs
Template
Delete
Cancel
Redeploy
Download
Refresh
Your deployment is complete
Deployment name : AccessConnectorCreate-20261104153747
Subscription : Azure subscription 1
Resource group : rg-sales-view-devtst
Start time : 11/4/2026, 3:38:45 PM
Correlation ID : 4b52ec0b-771a-47f6-ad46-54e66137c546
Deployment details
Next steps
Go to resource
Cost management
Get notified to stay within your budget and prevent unexpected charges on your bill.
Set up cost alerts >
Microsoft Defender for Cloud
Secure your apps and infrastructure
Go to Microsoft Defender for Cloud >
Free Microsoft tutorials
Start learning today >
Work with an expert
Azure experts are service provider partners who can help manage your assets on


Microsoft Azure
databricks
Search data, notebooks, recents, and more...
CTRL + P
dbw-sales-view-devtst
New
Workspace
Recents
Catalog
Jobs & Pipelines
Compute
Discover
Marketplace
SQL
SQL Editor
Queries
Dashboards
Genie Agents
Alerts
Query History
SQL Warehouses
Catalog Explorer › Credentials ›
cred_sales_view_adls
Overview
Usage
Permissions
Workspaces
Description
Add description
About this credential
Owner Shivam Kumar
Credential Type
Managed Identity
Purpose
STORAGE
Limit to read-only use
Disabled
Connector Id
/subscriptions/21ae3ec6-9aa4-4ccf-955a-144992377a54/resourceGroups/rg-sales-view-devtst/providers/Microsoft.Databricks/accessConnectors/ac-sales-view-devtst
User Assigned Managed Identity Id
Tags
Add tags


Microsoft Azure
databricks
Search data, notebooks, recents, and more...
Successful!
Your external location: ext_sales_view_bronze has been created
New
Workspace
Recents
Catalog
Jobs & Pipelines
Compute
Discover
Marketplace
SQL
SQL Editor
Queries
Dashboards
Genie Agents
Alerts
Query History
SQL Warehouses
Catalog Explorer › External Locations ›
ext_sales_view_bronze
Overview
Permissions
Browse
Workspaces
Description
Add description
About this external location
Owner Shivam Kumar
Credential
cred_sales_view_adls
URL
abfss://bronze@stsalesview20261104.dfs.core.windows.net/
Limit to read-only use
Disabled
Enable file events
Enabled
Resource group
rg-sales-view-devtst
Subscription ID
21ae3ec6-9aa4-4ccf-955a-144992377a54
Tags
Add tags
Fallback mode:

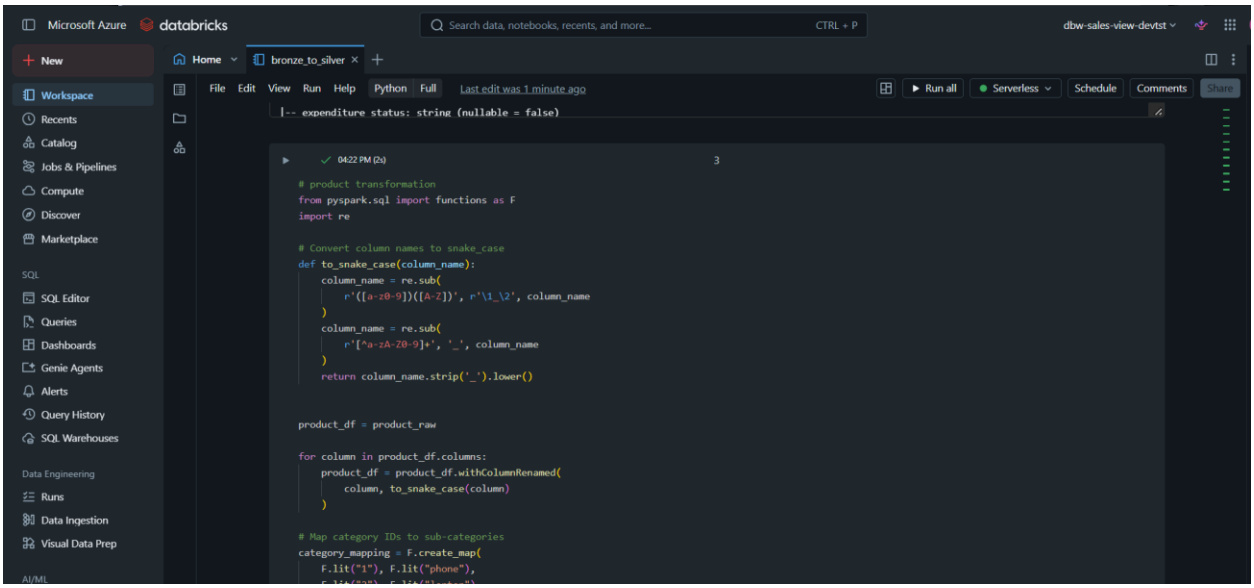

Microsoft Azure
databricks
Search data, notebooks, recents, and more...
CTRL + P
dbw-sales-view-devtst
New
Home
bronze_to_silver
File Edit View Run Help Python Full
Last edit was now
Run all
Serverless
Schedule
Comments
Share
Workspace
Recents
Catalog
Jobs & Pipelines
Compute
Discover
Marketplace
SQL
SQL Editor
Queries
Dashboards
Genie Agents
Alerts
Query History
SQL Warehouses
Data Engineering
Runs
Data Ingestion
Visual Data Prep
AI/ML
04:21 PM (5s)
2
Python
customer transformation
from pyspark.sql import functions as F
import re
Convert column names to snake_case
def to_snake_case(column_name):
column_name = re.sub(
r'([a-z0-9])([A-Z])', r'\1_\2', column_name
)
column_name = re.sub(
r'[^a-zA-Z0-9]+', '_', column_name
)
return column_name.strip('_').lower()
customer_df = customer_raw
for column in customer_df.columns:
customer_df = customer_df.withColumnRenamed(
column, to_snake_case(column)
)
Split customer name
customer_df = (
customer_df
.withColumn(
"first name",

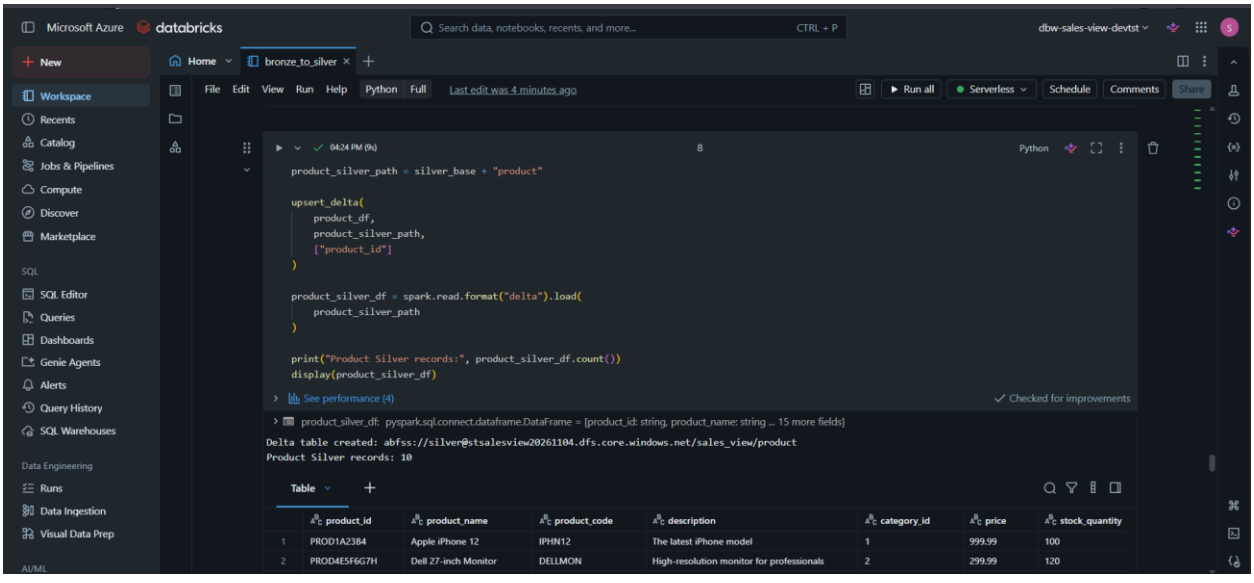

Microsoft Azure
databricks
Search data, notebooks, recents, and more...
CTRL + P
dbw-sales-view-devtst
New
Home
bronze_to_silver
File Edit View Run Help Python Full
Last edit was 3 minutes ago
Run all
Serverless
Schedule
Comments
Share
Workspace
Recents
Catalog
Jobs & Pipelines
Compute
Discover
Marketplace
SQL
SQL Editor
Queries
Dashboards
Genie Agents
Alerts
Query History
SQL Warehouses
Data Engineering
Runs
Data Ingestion
Visual Data Prep
AI/ML
04:23 PM (1s)
6
Python
from delta.tables import DeltaTable
from pyspark.sql import functions as F
silver_base = (
"abfss://silver@stsalesview20261104.dfs.core.windows.net/"
"sales_view/"
)
def upsert_delta(df, table_path, merge_keys):
Remove duplicate records using the merge key
source_df = df.dropDuplicates(merge_keys)
if DeltaTable.isDeltaTable(spark, table_path):
delta_table = DeltaTable.forPath(spark, table_path)
condition = " AND ".join(
[f"target.`{key}` = source.`{key}`"
for key in merge_keys]
)
update_values = {
col: f"source.`{col}`"
for col in source_df.columns
if col not in merge_keys


Microsoft Azure
databricks
Search data, notebooks, recents, and more...
CTRL + P
dbw-sales-view-devtst
New
Home
bronze_to_silver
File Edit View Run Help Python Full
Last edit was 4 minutes ago
Run all
Serverless
Schedule
Comments
Share
Workspace
Recents
Catalog
Jobs & Pipelines
Compute
Discover
Marketplace
SQL
SQL Editor
Queries
Dashboards
Genie Agents
Alerts
Query History
SQL Warehouses
Data Engineering
Runs
Data Ingestion
Visual Data Prep
AI/ML
04:24 PM (9s)
7
Python
customer_silver_path = silver_base + "customer"
upsert_delta(
customer_df,
customer_silver_path,
["customer_id"]
)
customer_silver_df = spark.read.format("delta").load(
customer_silver_path
)
print("Customer Silver records:", customer_silver_df.count())
display(customer_silver_df)
See performance (4)
Checked for improvements
customer_silver_df: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, name: string ... 17 more fields]
Delta table created: abfss://silver@stsalesview20261104.dfs.core.windows.net/sales_view/customer
Customer Silver records: 29
Table
customer_id
name
email_id
address
gender
age
16 amxdi438 Diane Henson dianehenson438@wingacademy.com 86382 Baker Ports Apt. 074 Cookfurt, NM 54313 F 74 20
17 amxni473 Nicole Lara nicolelara473@slingacademy.com 9822 Lisa Courts Johnhaven, IL 80438 F 77 20

5) Product File

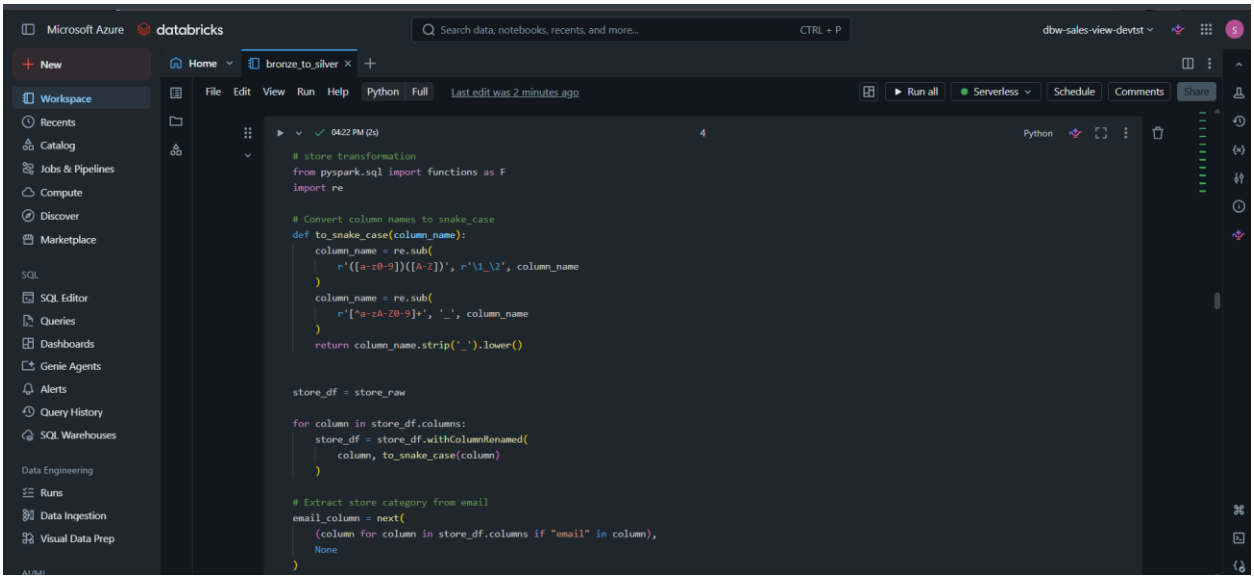
- All the column header shlould be in snake case in lower case (use same UDF Function)
- Create a column sub_category (Use Category columns id category_id=1, "phone"; 2, "laptop"; 3,"playstation"; 4,"e-device"
- Write based on upsert [table_name: product](in silver layer path is silver/sales_view/tablename/{delta pearquet}

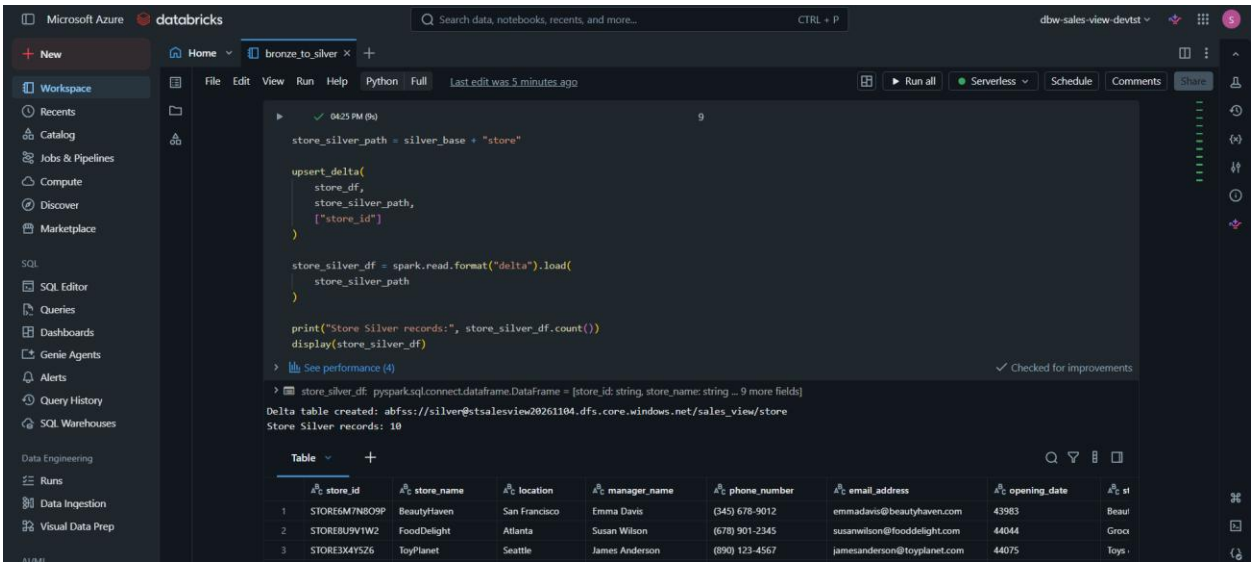




6) Store

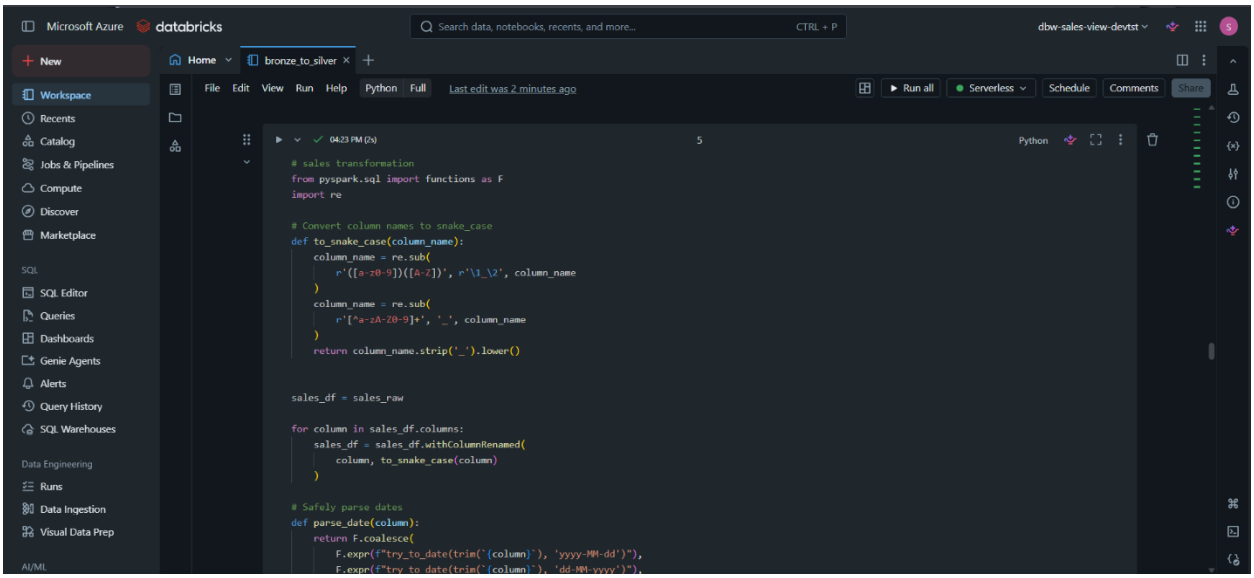
- Read the data make sure header shlould be in snake case in lower case (use same UDF Function)
- Create a store category columns and the value is exatracted from email Eg: "electromart" from johndoe@electromart.com
- created_at, updated_at date as yyyy-MM-dd format
- Write based on upsert [table_name: store] (in silver layer path is silver/sales_view/tablename/{delta pearquet}

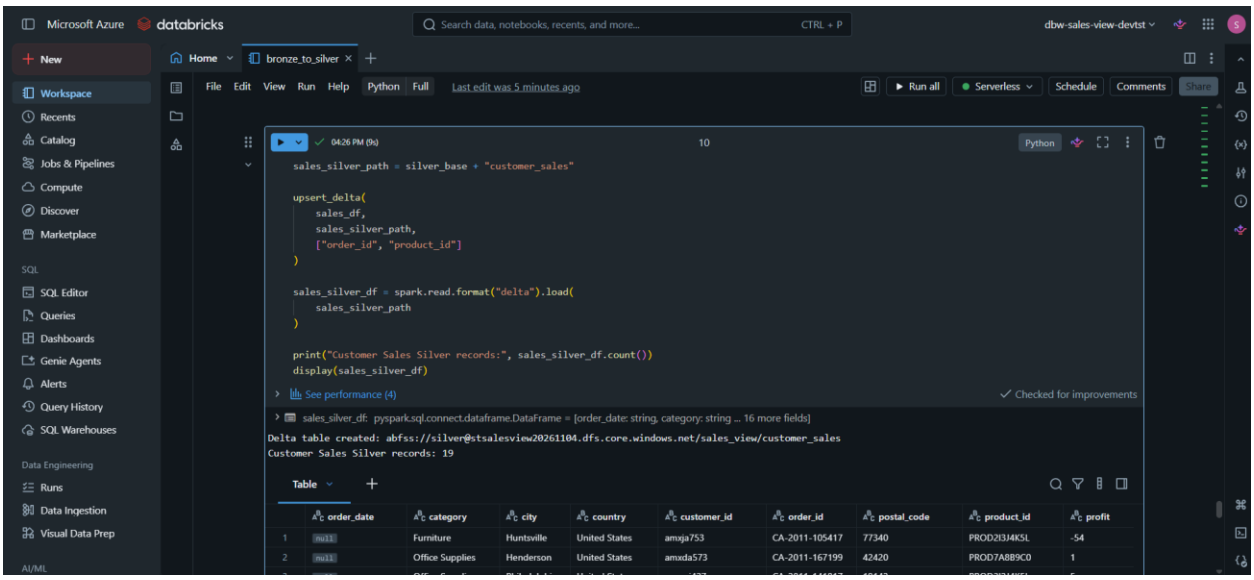




7) Sales

- Read the data make sure header shlould be in snake case in lower case (use same UDF Function)
- Write based on upsert [table_name: customer_sales] (in silver layer path is silver/sales_view/tablename/{delta pearquet}





8) In gold layer

- using product and store table get the below data

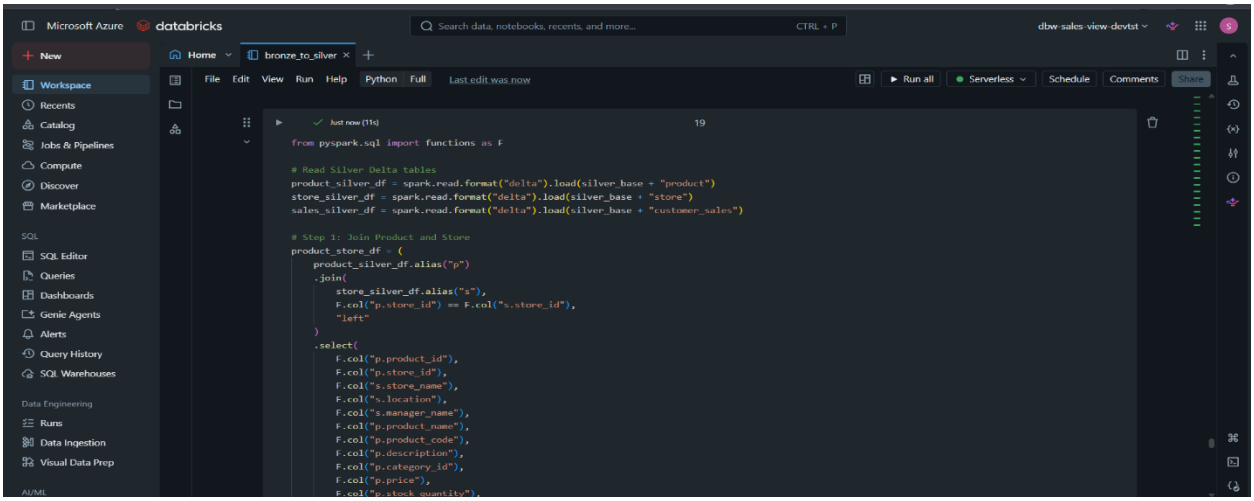
1. store_id,store_name,location,manager_name,product_name,product_code,description,category_id,price,stock_quantity,supplier_id,product_created_at,product_updated_at,image_url,weight,expiry_date,is_active,tax_rate.

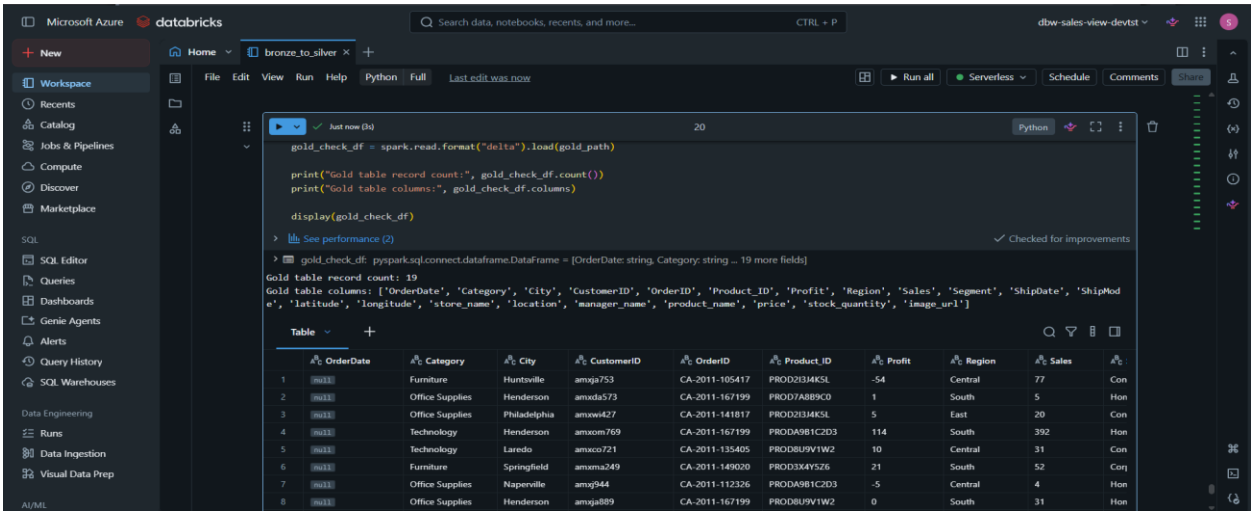
- Read the delta table (using UDF functions)

- Using the above data & customer_sales and get the below data

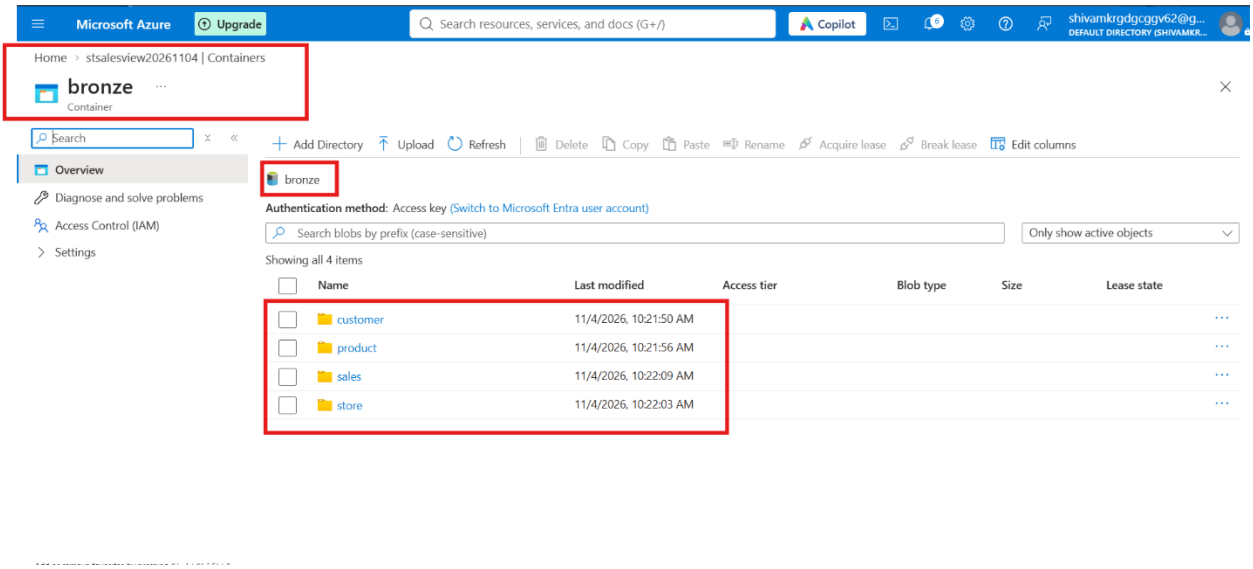
2. OrderDate,Category,City,CustomerID,OrderID,Product ID, Profit,Region, Sales,Segment,ShipDate,ShipMode,latitude,longitude,store_name,location,manager_name,product_name,price,stock_quantity,image_url

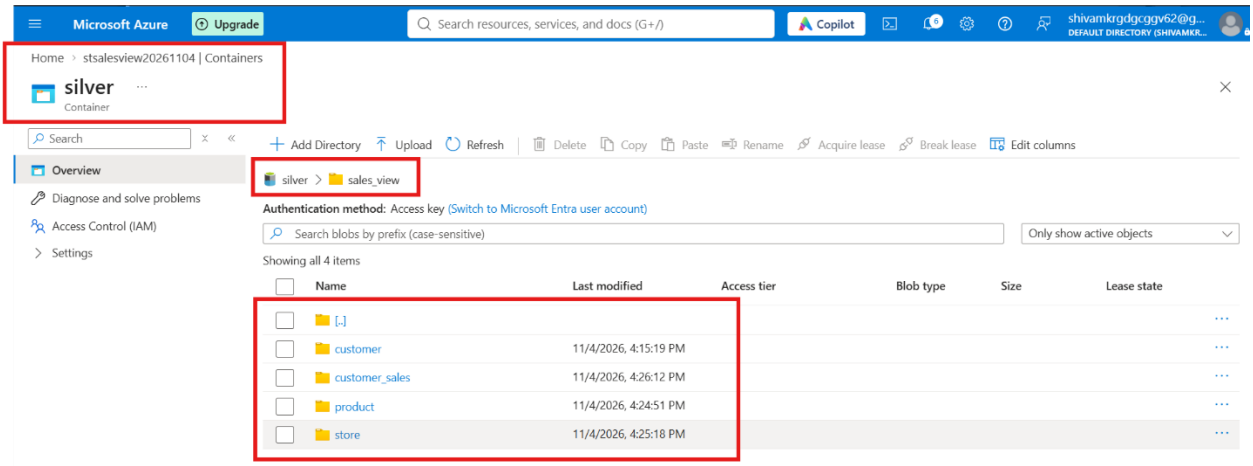
- Write based on overwrite (table_name : StoreProductSalesAnalysis)(in gold layer path is gold/sales_view/tablename/{delta pearquet}

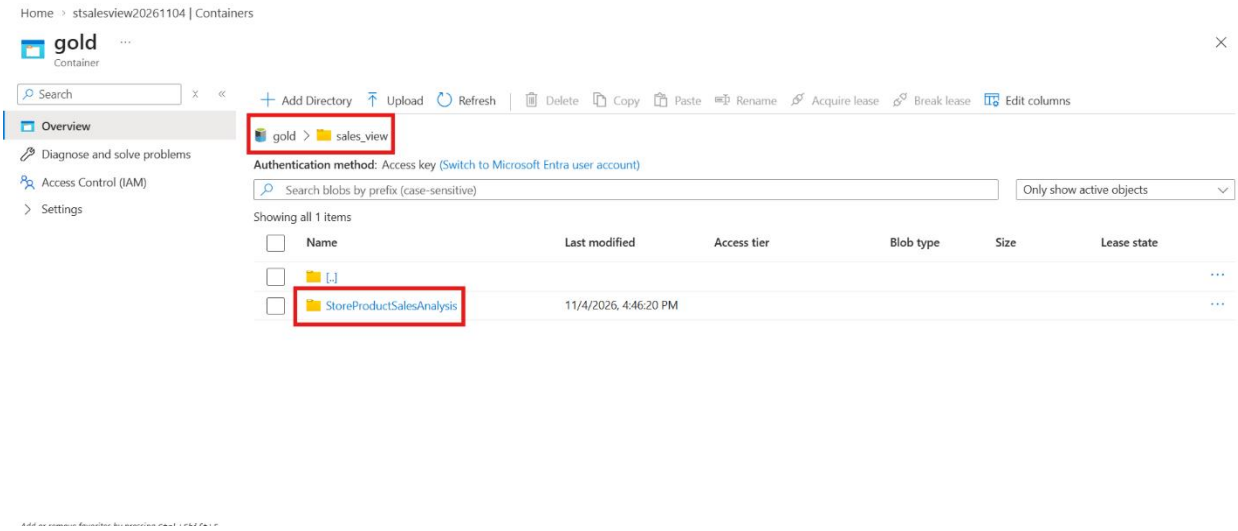


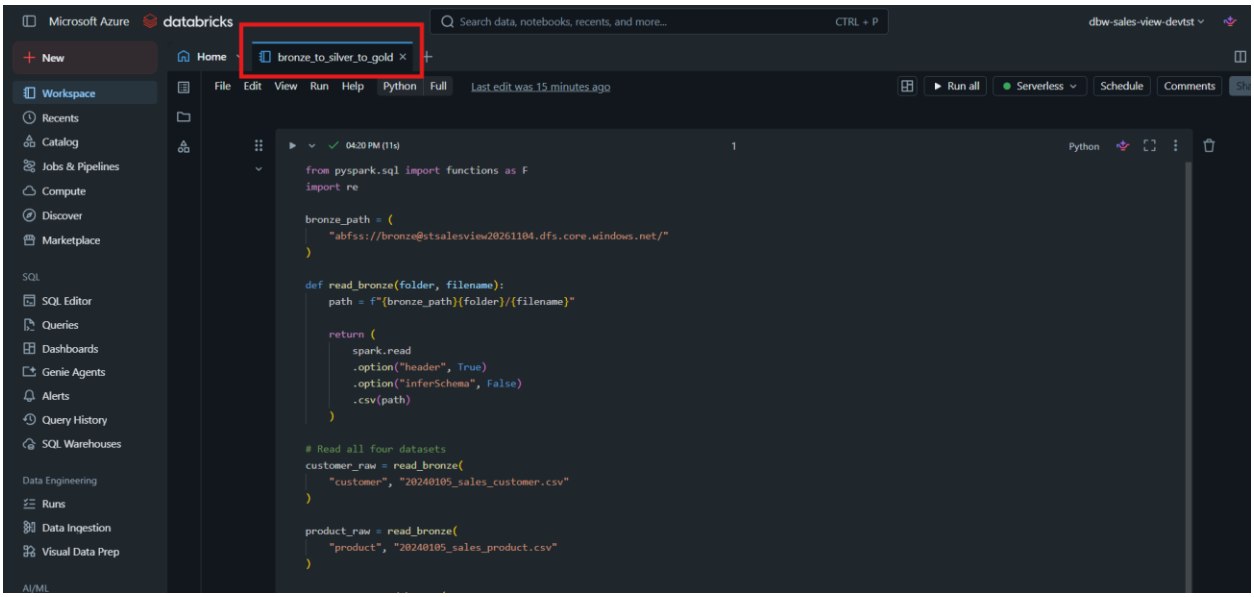


So, at the end below are things you need to have. -









ADF and Azure Databricks – Practical Use Case Documentation

1. What is Data Profiling and How Is It Done?

1.1 Definition

Data profiling is the process of examining and analyzing data to understand its structure, quality, completeness, consistency, and reliability.

It helps identify data quality issues before the data is used for transformation, reporting, or business analysis.

The primary objective of data profiling is to ensure that the data is accurate, consistent, complete, and suitable for further processing.

1.2 How Is Data Profiling Performed?

Data profiling generally involves the following steps:

1. Structural Analysis
    - Identify the number of rows and columns.
    - Understand column names and data types.
    - Identify the structure of the source datasets.
2. Completeness Analysis
    - Identify NULL and blank values.

- Check whether mandatory fields contain missing values.
- Measure the completeness of important columns.

3. Uniqueness Analysis
    - Identify duplicate records.
    - Check duplicate primary or business keys.
    - Verify whether unique identifiers are maintained.
4. Validity Analysis
    - Validate email addresses, dates, numeric values, and categorical fields.
    - Identify invalid or incorrectly formatted values.
    - Verify whether data follows expected business rules.
5. Consistency Analysis
    - Check whether the same information follows consistent formats across datasets.
    - Validate relationships between different tables.
    - Identify mismatched foreign keys and reference values.
6. Data Distribution Analysis
    - Understand distinct values and their frequency.
    - Identify unexpected values and unusual distributions.
    - Analyze numerical columns such as sales, expenditure, and product prices.
7. Data Quality Reporting
    - Document identified issues.
    - Record the actions taken to address them.
    - Determine whether the data is suitable for downstream processing.

---

2. Data Profiling Steps Performed for the Provided Data

The assignment contains four source datasets:

- Customer
- Product
- Store
- Sales

These datasets were stored in Azure Data Lake Storage Gen2 and processed using Azure Data Factory and Azure Databricks.

2.1 Source Dataset Analysis

The initial source datasets were inspected to understand their structure, columns, and record counts.

Dataset	Number of Records	Profiling Focus
Customer	29	Customer IDs, names, email addresses, gender, joining dates and expenditure
Product	10	Product IDs, categories, prices, stock quantities and Store IDs
Store	10	Store IDs, names, locations, email addresses and dates
Sales	19	Order IDs, Product IDs, Customer IDs, sales, profit and order dates

2.2 Structural Profiling

The following activities were performed:

- Examined the schema of all four datasets.
- Identified column names and source data formats.
- Loaded CSV data into PySpark DataFrames.
- Initially treated source columns as strings to avoid incorrect automatic type conversion.
- Examined the total number of records and columns.

2.3 Customer Data Profiling

The Customer dataset was examined for:

- Customer identification and name fields.
- Email address structure and domain extraction.
- Gender values and standardization requirements.
- Joining Date format consistency.
- Expenditure values and business classification.

Transformations performed:

- Converted column names into lowercase snake_case.
- Split the Name column into first_name and last_name.
- Extracted the email domain.
- Standardized gender values into M and F.
- Separated joining date and joining time.
- Standardized dates into yyyy-MM-dd.
- Classified expenditure as MINIMUM when spent is below 200 and MAXIMUM otherwise.

2.4 Product Data Profiling

The Product dataset was examined for:

- Product ID uniqueness.
- Product category values.
- Product prices and stock quantities.
- Product creation, update and expiry dates.
- Store ID relationships.

Transformations performed:

- Standardized column names into lowercase snake_case.
- Created sub_category based on category_id:

    - 1 – phone
    - 2 – laptop
    - 3 – playstation
    - 4 – e-device
- Standardized applicable date columns into yyyy-MM-dd.

2.5 Store Data Profiling

The Store dataset was examined for:

- Store ID and store name.
- Store locations and manager details.
- Email address structure.
- Created and updated date formats.
- Relationships with Product records.

Transformations performed:

- Standardized column names.
- Extracted store_category from the email domain.
- Standardized created_at and updated_at dates into yyyy-MM-dd.

2.6 Sales Data Profiling

The Sales dataset was examined for:

- Order ID and Product ID.
- Customer ID.
- Sales and profit values.
- Order and shipping dates.
- Region, segment, city and shipping mode.

Transformations performed:

- Standardized column names into lowercase snake_case.
- Standardized order_date and ship_date into yyyy-MM-dd.

- Prepared the dataset for integration with Product and Store information.

2.7 Relationship and Referential Integrity Profiling

During the Gold-layer implementation, an important data quality issue was identified.

The Product dataset contained Store IDs that did not match any Store IDs in the Store dataset.

For example:

- Product referenced STORE3P4Q5R6S.
- Store contained STORE3X4Y5Z6 instead.

The Product–Store join returned zero matching records.

However, the Product–Sales join successfully matched 19 records.

Action taken:

A LEFT JOIN was used between Product and Store, followed by a LEFT JOIN with Sales.

This preserved all 19 Sales records while keeping unmatched Store details as NULL.

The mismatch was documented as a source-data quality issue rather than introducing artificial ID mappings.

Note: Structural profiling, record counts, schema inspection, and relationship checks were performed during implementation. Detailed NULL-frequency, duplicate-key, and statistical distribution reports should be run separately if numerical results for those checks are required. No unverified profiling statistics are assumed in this report.
